

Jenkins CI/CD Architectures (Interview Notes)

1. Monorepo (Single Pipeline + Multiple Services)

Definition:

Monorepo architecture me multiple microservices ek hi Git repository ke andar hote hain aur ek hi Jenkins pipeline un sab services ko build, test aur deploy karti hai — usually parallel execution ke through.

Example:

Repo B:

```
├─ payments
├─ transfers
└─ loan
```

Jenkins:

- ONE pipeline
- payments + transfers + loan (parallel build & deploy)

Groovy (Scripted Pipeline Example):

```
node {
    def services = ['payments', 'transfers', 'loan']
    def branches = [:]

    stage('Parallel Build') {
        for (s in services) {
            branches[s] = {
                dir(s) {
                    sh 'mvn clean package'
                    sh "docker build -t myrepo/${s}:$BUILD_NUMBER ."
                    sh "kubectl apply -f k8s/"
                }
            }
        }
        parallel branches
    }
}
```

Key Points:

- One repository

- One Jenkins pipeline
 - Multiple services handled together
 - Parallel execution possible
-

2. Multibranch Pipeline (Branch-based CI/CD)

Definition:

Multibranch pipeline Jenkins ka ek feature hai jisme har Git branch ke liye alag pipeline automatically create hota hai, aur har branch ka apna independent CI/CD run hota hai.

Example:

Git Repo:

```
main → pipeline run  
dev → pipeline run  
feature/login → pipeline run
```

Groovy (Scripted Pipeline Example):

```
node {  
    stage('Checkout') {  
        checkout scm  
    }  
  
    stage('Build') {  
        sh 'mvn clean package'  
    }  
  
    stage('Test') {  
        sh 'mvn test'  
    }  
  
    stage('Deploy') {  
        if (env.BRANCH_NAME == 'main') {  
            sh 'kubectl apply -f k8s/prod'  
        } else if (env.BRANCH_NAME == 'dev') {  
            sh 'kubectl apply -f k8s/dev'  
        }  
    }  
}
```

Key Points:

- Based on Git branches
- Each branch = separate pipeline run
- Used for PR, dev, QA workflows
- Automatic pipeline creation

3. Multi-Microservice Pipeline (Separate Pipelines)

Definition:

Is architecture me har microservice ka apna alag Jenkins job/pipeline hota hai, aur sab services independently build, test aur deploy hoti hain. Ek orchestrator pipeline in sab ko trigger kar sakta hai.

Example:

```
payments-job    → pipeline
transfers-job   → pipeline
loan-job        → pipeline
```

Optional Orchestrator:

```
Orchestrator Job → triggers all 3 jobs in parallel
```

Groovy (Orchestrator Scripted Pipeline Example):

```
node {
    def services = ['payments','transfers','loan']
    def jobs = [:]

    stage('Trigger All Services') {
        for (s in services) {
            jobs[s] = {
                catchError(buildResult: 'SUCCESS', stageResult: 'FAILURE') {
                    build job: "${s}-job", wait: true
                }
            }
        }
        parallel jobs
    }
}
```

Key Points:

- One service = one pipeline
 - Fully isolated CI/CD
 - Highly scalable architecture
 - Orchestrator optional
-

Comparison Summary

Type	Based On	Pipeline Style
Monorepo	One repo, many services	Single pipeline
Multibranch	Git branches	Multiple pipelines per branch
Multi-microservice	Service-wise separation	Multiple independent pipelines

Quick Memory Trick

- Monorepo → One repo, one pipeline, many services
 - Multibranch → One repo, many branches, many pipelines
 - Multi-service → Many services, many pipelines, fully isolated
-

Final Note

In real-world DevOps systems, companies often combine these patterns depending on scale, team structure, and deployment strategy.